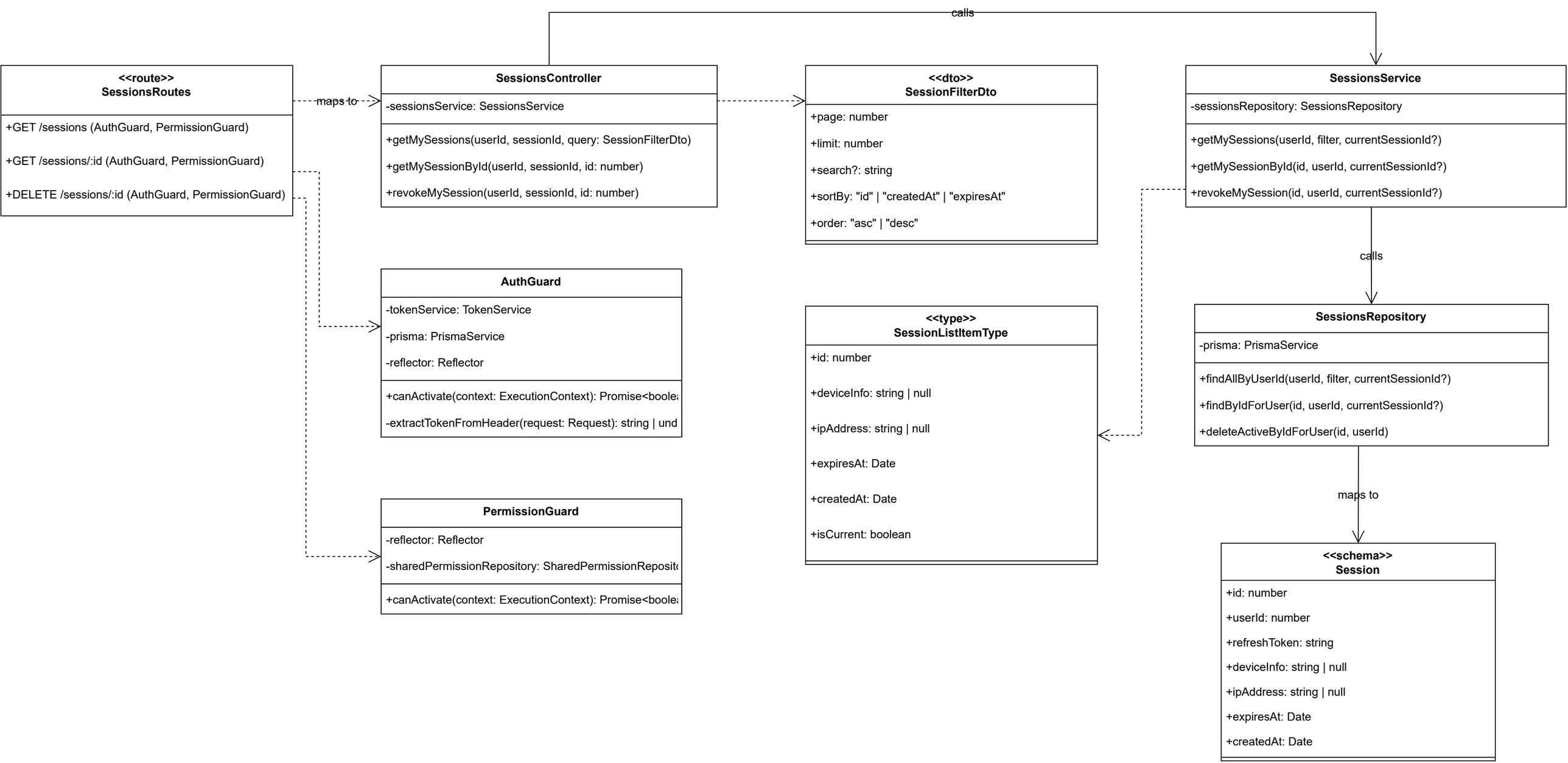
calls
<<route>>
SessionsRoutes
+GET /sessions (AuthGuard, PermissionGuard)
+GET /sessions/:id (AuthGuard, PermissionGuard)
+DELETE /sessions/:id (AuthGuard, PermissionGuard)
maps to
SessionsController
-sessionsService: SessionsService
+getMySessions(userId, sessionId, query: SessionFilterDto)
+getMySessionById(userId, sessionId, id: number)
+revokeMySession(userId, sessionId, id: number)
<<dto>>
SessionFilterDto
+page: number
+limit: number
+search?: string
+sortBy: "id" | "createdAt" | "expiresAt"
+order: "asc" | "desc"
SessionsService
-sessionsRepository: SessionsRepository
+getMySessions(userId, filter, currentSessionId?)
+getMySessionById(id, userId, currentSessionId?)
+revokeMySession(id, userId, currentSessionId?)
calls
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boole
-extractTokenFromHeader(request: Request): string | und
<<type>>
SessionListItemType
+id: number
+deviceInfo: string | null
+ipAddress: string | null
+expiresAt: Date
+createdAt: Date
+isCurrent: boolean
SessionsRepository
-prisma: PrismaService
+findAllByUserId(userId, filter, currentSessionId?)
+findByIdForUser(id, userId, currentSessionId?)
+deleteActiveByIdForUser(id, userId)
maps to
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionReposit
+canActivate(context: ExecutionContext): Promise<boole
<<schema>>
Session
+id: number
+userId: number
+refreshToken: string
+deviceInfo: string | null
+ipAddress: string | null
+expiresAt: Date
+createdAt: Date